Lecture 28

Fundamentals of Numerical Analysis

C. Mou

Department of Mathematics and Statistics
Utah State University

Fall 2026

Last class and last last class

We get

$$A = \begin{bmatrix} \vdots & \vdots & \vdots \\ u_1 & \cdots & u_m \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \sigma_1 & & & \\ & \ddots & & \\ & & \sigma_r & \\ & & & \ddots \\ & & & & 0 \end{bmatrix} \begin{bmatrix} \cdots & v_1^T & \cdots \\ \cdots & \vdots & \cdots \\ \cdots & v_n^T & \cdots \end{bmatrix}$$

Today

1. SVD in image processing
2. Least square

SVD: who cares?

How can we actually *use* $A = USV^T$? We can use this to represent A with far fewer entries...

Notice what $A = USV^T$ looks like:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T + 0 u_{r+1} v_{r+1}^T + \cdots + 0 u_p v_p^T$$

This is easily truncated to

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots + \sigma_r u_r v_r^T$$

Motivation for Low-Rank Approximation

Many real-world matrices are *approximately low rank*:

$$A \approx A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^T, \qquad k \ll \min(m, n)$$

- Data often lie near a low-dimensional subspace (e.g. correlated features).
- A_k captures the most energetic directions of A.
- Greatly reduces storage and computation while preserving structure.

Example applications:

- Image compression (store top singular triplets).
- Recommendation systems (latent factors).
- Reduced-order models for PDEs or dynamical systems.

Energy and Error in Low-Rank SVD

The total energy (Frobenius norm squared) of A is

$$\|A\|_F^2 = \sum_{i=1}^{r} \sigma_i^2.$$

Keeping k modes captures fraction

$$E_k = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2}.$$

- Choose k such that $E_k \geqslant 0.95$ or 0.99.
- Error in 2-norm: $\|A - A_k\|_2 = \sigma_{k+1}$.
- Error in Frobenius norm: $\|A - A_k\|_F^2 = \sum_{i=k+1}^{r} \sigma_i^2$.

Interpretation: each additional singular value adds an orthogonal "direction of variance."

Geometric Picture of Low-Rank Approximation

- A maps the unit sphere to an ellipsoid with axes σ_i.
- Truncating SVD removes the smallest axes $\sigma_{k+1}, \ldots$, flattening the ellipsoid into a lower-dimensional subspace.
- The approximation A_k projects data onto the span of the top k left singular vectors U_k.

$$A \approx U_k \Sigma_k V_k^T, \quad A_k x = U_k \Sigma_k (V_k^T x)$$

Meaning:

- V_k encodes *dominant input directions*.
- U_k encodes *dominant output directions*.

Why SVD for Images?

- A grayscale image is just a big table of numbers (pixels) $\Rightarrow$ a matrix A.
- **Goal:** keep the important visual information using fewer numbers (*compression*).
- **SVD idea (informal):** A can be written as a sum of simple "layers" (patterns).
- If we keep only the first k layers, we get a good approximation of the image.

Key benefit: smaller storage and faster processing, with little loss in quality (for reasonable k).

What Does "Rank-k" Mean?

- SVD writes A as a sum of pieces: $A = \sigma_1 u_1 v_1^T + \cdots + \sigma_r u_r v_r^T$.
- The numbers $\sigma_1 \geqslant \sigma_2 \geqslant \cdots$ measure each piece's importance.
- **Rank-k image:** keep only the first k pieces:

$$A_k = \sigma_1 u_1 v_1^T + \cdots + \sigma_k u_k v_k^T.$$

- Bigger $k \Rightarrow$ better quality (and larger file).

Rule of thumb: increase k until the image looks good enough.

How Much Are We Saving?

- Original image: $m \times n$ pixels $\Rightarrow$ stores mn numbers.
- Rank-k version stores about $k(m+n+1)$ numbers (much smaller when $k \ll \min(m,n)$).

Example: 512×512 image

$$\text{original} = 262{,}144 \quad \text{vs.} \quad \text{rank-40} \approx 40(512+512+1) = 40{,}960.$$

That's about **6.4**$\times$ **less** numbers.

What Does “Energy” Mean?

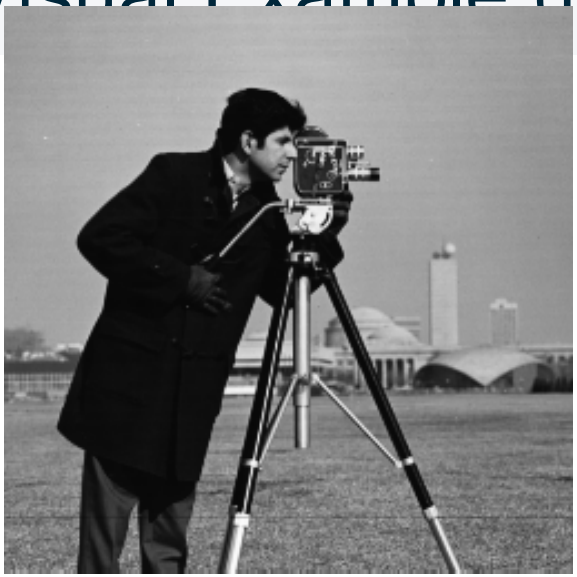
- The σ_i tell us how much each layer contributes.
- Define **cumulative energy** after k layers:

$$E_k = \frac{\sigma_1^2 + \cdots + \sigma_k^2}{\sigma_1^2 + \cdots + \sigma_r^2}.$$

- Pick the smallest k with E_k around 95%–99%.

Translation: keep enough layers to capture most of the image’s detail.

Visual Example (fill these figures)



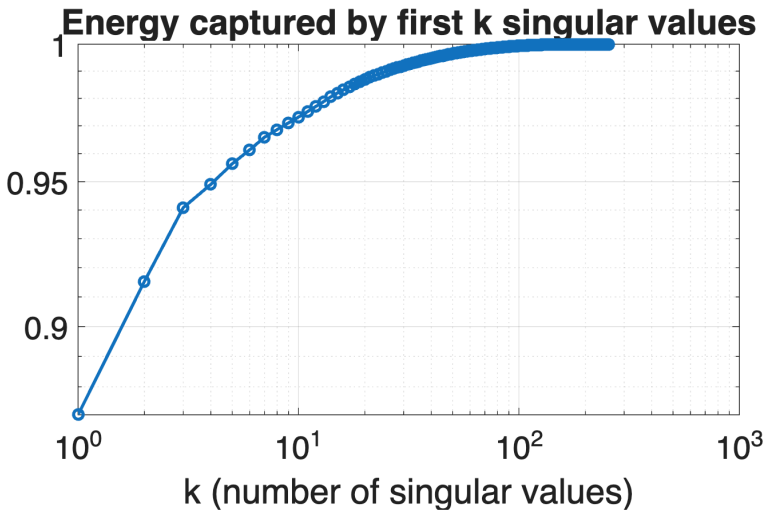
Original



Rank-10



Rank-40



Singular value



Rank-80

Minimal Code to Generate the Figures (MATLAB)

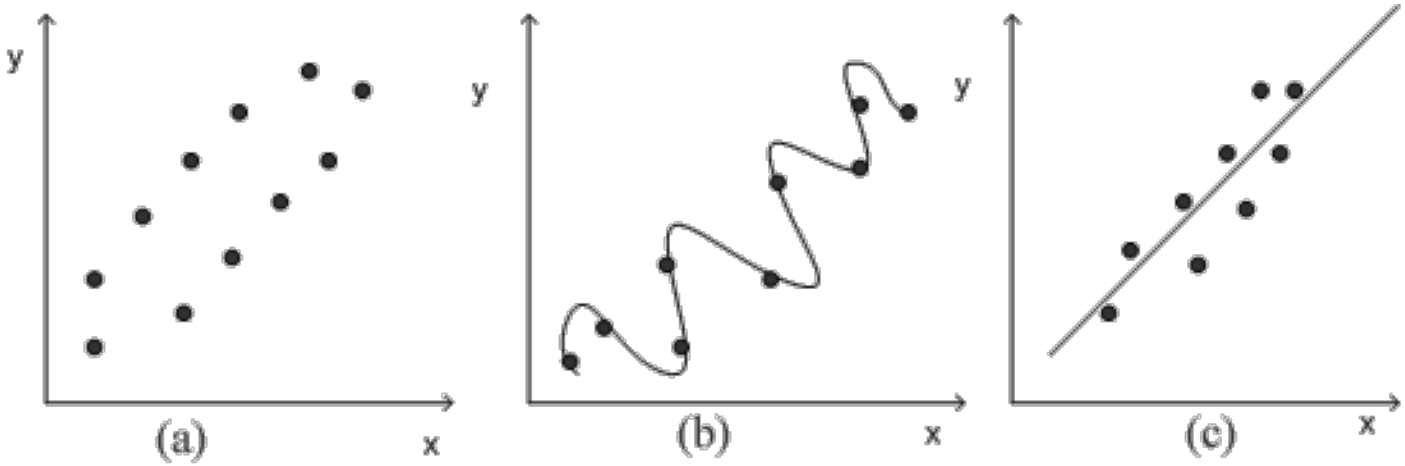
```
I = im2double(imread('cameraman.tif'));   % grayscale image
[U,S,V] = svd(I,'econ'); s = diag(S);     % SVD and singular values

% helper to save rank-k reconstruction
save_k = @(k,name) imwrite(U(:,1:k)*S(1:k,1:k)*V(:,1:k)', ...
                           fullfile('figs',name));
if ~exist('figs','dir'), mkdir figs; end
imwrite(I, 'figs/svd_img_original.png');
save_k(20,'svd_img_k20.png'); save_k(60,'svd_img_k60.png');
save_k(100,'svd_img_k100.png');

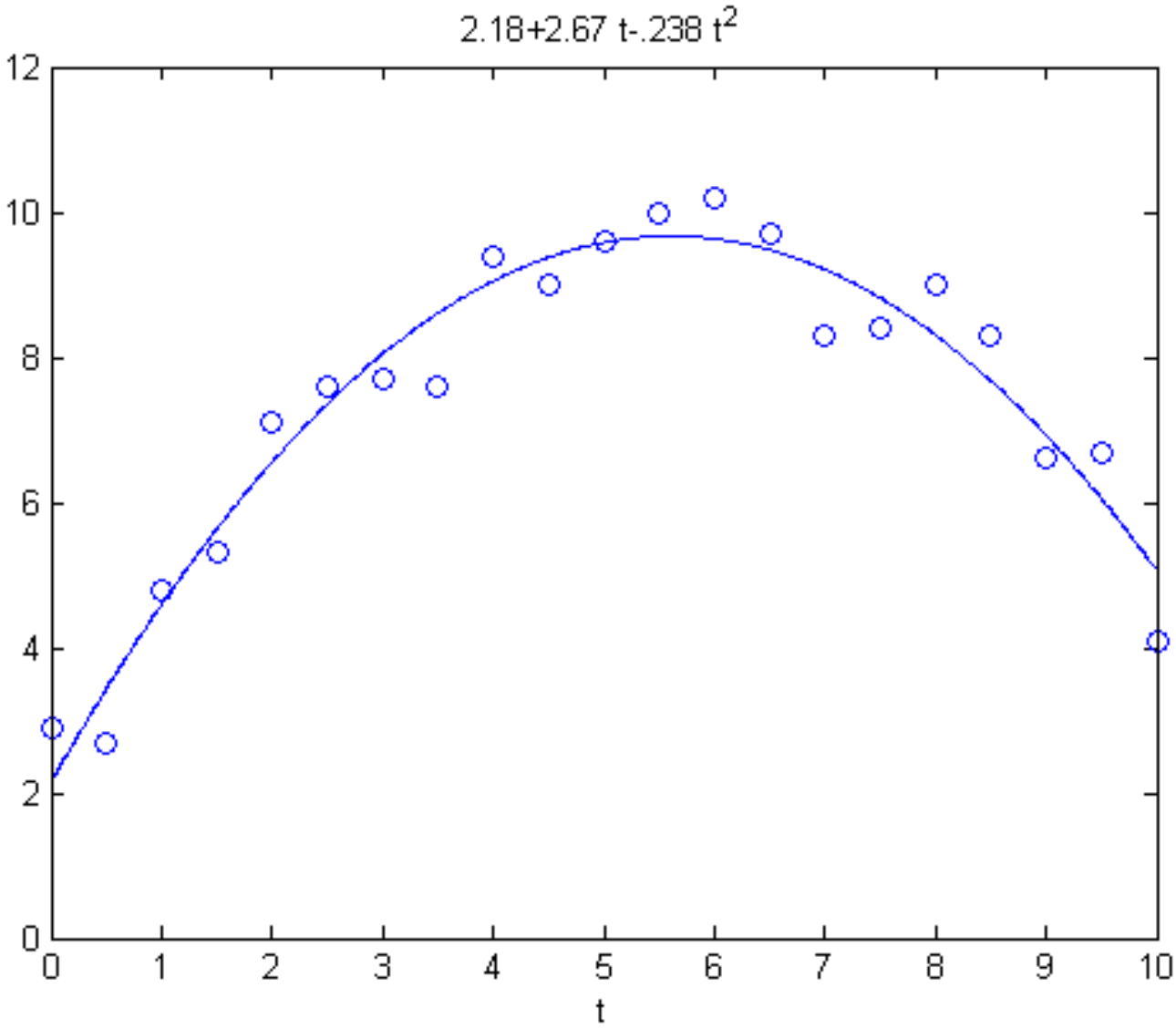
Ek = cumsum(s.^2)/sum(s.^2);                % cumulative energy
f = figure('visible','off'); plot(Ek,'-o'); grid on
xlabel('k'); ylabel('Energy E_k'); ylim([0 1])
print(f, 'figs/svd_sigma_energy.png','-dpng','-r200'); close(f)
```

Fitting data

Recall in interpolation we wanted to find a curve that went through *all* of the data points.
Suppose we are given the data $\{(x_1, y_1), ..., (x_n, y_n)\}$ and we want to find a curve that *best fits* the data.



Fitting curves



Fitting a line

Given n data points $\{(x_1, y_i), ..., (x_n, y_n)\}$ find a and b such that

$$y_i = ax_i - b \quad \forall i \in [1, n].$$

In matrix form, find a and b that solves

$$\begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}$$

Systems with more equations than unknowns are called **overdetermined**

Overdetermined Systems

If A is an $m \times n$ matrix, then in general, an $m \times 1$ vector b may not lie in the column space of A. Hence $Ax = b$ may not have an exact solution.

Definition

The **residual** vector is

$$r = b - Ax.$$

The **least squares** solution is given by minimizing the square of the residual in the 2-norm.

Normal equations

Writing $r = (b - Ax)$ and substituting, we want to find an x that minimizes the following function

$$\phi(x) = \|r\|_2^2 = r^T r = (b - Ax)^T (b - Ax) = b^T b - 2x^T A^T b + x^T A^T Ax$$

From calculus we know that the minimizer occurs where $\nabla\phi(x) = 0$.

The derivative is given by

$$\nabla\phi(x) = -2A^T b + 2A^T Ax = 0$$

Definition

The system of **normal equations** is given by

$$A^T Ax = A^T b.$$

Solving normal equations

Since the normal equations forms a symmetric system, we can solve by computing the Cholesky factorization

$$A^TA = LL^T$$

and solving $Ly = A^Tb$ and $L^Tx = y$. Consider

$$A = \begin{bmatrix} 1 & 1 \\ \epsilon & 0 \\ 0 & \epsilon \end{bmatrix}$$

where $0 < \epsilon < \sqrt{\epsilon_{mach}}$. The normal equations for this system is given by

$$A^TA = \begin{bmatrix} 1+\epsilon^2 & 1 \\ 1 & 1+\epsilon^2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

Cholesky Factorization (in brief)

For any symmetric matrix M, the Cholesky factorization represents it as

$$M = LL^T, \qquad \text{where } L \text{ is a lower triangular matrix.}$$

Normal equations: conditioning

The normal equations tend to worsen the condition of the matrix.

Theorem

$$cond(A^T A) = (cond(A))^2$$

```
>> A = rand(10,10);
>> cond(A)
   43.4237
>> cond(A'*A)
   1.8856e+03
```

How can we solve the least squares problem without squaring the condition of the matrix?

Other approaches

- SVD - singular value decomposition
  - For $A \in \mathbb{R}^{m \times n}$, factor $A = USV^T$ where
    - U is an $m \times m$ orthogonal matrix
    - V is an $n \times n$ orthogonal matrix
    - S is an $m \times n$ diagonal matrix whose elements are the singular values.

- QR factorization.
  - For $A \in \mathbb{R}^{m \times n}$, factor $A = QR$ where
    - Q is an $m \times m$ orthogonal matrix
    - R is an $m \times n$ upper triangular matrix (since R is an $m \times n$ upper triangular matrix we can write $R = \begin{bmatrix} R' \\ 0 \end{bmatrix}$ where R is $n \times n$ upper triangular and 0 is the $(m-n) \times n$ matrix of zeros)

Orthogonal matrices

Definition

A matrix Q is orthogonal if

$$Q^T Q = Q Q^T = I$$

Orthogonal matrices preserve the Euclidean norm of any vector v,

$$\|Qv\|_2^2 = (Qv)^T (Qv) = v^T Q^T Q v = v^T v = \|v\|_2^2.$$

Using SVD for least squares

Recall that a singular value decomposition is given by

$$A = \begin{bmatrix} \vdots & \vdots & \vdots \\ u_1 & \cdots & u_m \\ \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} \sigma_1 & & & & \\ & \ddots & & & \\ & & \sigma_r & & \\ & & & \ddots & \\ & & & & 0 \end{bmatrix} \begin{bmatrix} \cdots & v_1^T & \cdots \\ \cdots & \vdots & \cdots \\ \cdots & v_n^T & \cdots \end{bmatrix}$$

where σ_i are the singular values.

Using SVD for least squares

Assume that A has rank k (and hence k nonzero singular values σ_i) and recall that we want to minimize

$$||r||_2^2 = ||b - Ax||_2^2.$$

Substituting the SVD for A we find that

$$||r||_2^2 = ||b - Ax||_2^2 = ||b - USV^T x||_2^2$$

where U and V are orthogonal and S is diagonal with k nonzero singular values.

$$||b - USV^T x||_2^2 = ||U^T b - U^T USV^T x||_2^2 = ||U^T b - SV^T x||_2^2$$

Using SVD for least squares

Let $c = U^T b$ and $y = V^T x$ (and hence $x = Vy$) in $||U^T b - SV^T x||_2^2$. We now have

$$||r||_2^2 = ||c - Sy||_2^2$$

Since S has only k nonzero diagonal elements, we have

$$||r||_2^2 = \sum_{i=1}^{k} (c_i - \sigma_i y_i)^2 + \sum_{i=k+1}^{n} c_i^2$$

which is minimized when $y_i = \frac{c_i}{\sigma_i}$ for $1 \leqslant i \leqslant k$.

Using SVD for least squares

Theorem

Let A be an $m \times n$ matrix of rank r and let $A = USV^T$, the singular value decomposition. The least squares solution of the system $Ax = b$ is

$$x = \sum_{i=1}^{r} (\sigma_i^{-1} c_i) v_i$$

where $c_i = u_i^T b$.

Using QR factorization for least squares

Now that we know orthogonal matrices preserve the euclidean norm, we can apply orthogonal matrices to the residual vector without changing the norm of the residual.

$$\|r\|_2^2 = \|b - Ax\|_2^2 = \left\| b - Q \begin{bmatrix} R \\ 0 \end{bmatrix} x \right\|_2^2 = \left\| Q^T b - Q^T Q \begin{bmatrix} R \\ 0 \end{bmatrix} x \right\|_2^2 = \left\| Q^T b - \begin{bmatrix} R \\ 0 \end{bmatrix} x \right\|_2^2$$

If $Q^T b = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$ and $x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$ then

$$\left\| Q^T b - \begin{bmatrix} R \\ 0 \end{bmatrix} \right\|_2^2 = \left\| \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} - \begin{bmatrix} Rx_1 \\ 0 \end{bmatrix} \right\|_2^2 = \left\| \begin{bmatrix} c_1 - Rx_1 \\ c_2 \end{bmatrix} \right\|_2^2 = \|c_1 - Rx_1\|_2^2 + \|c_2\|_2^2$$

Hence the least squares solution is given by solving $\begin{bmatrix} R \\ 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix}$. We can solve $Rx_1 = c_1$ using back substitution and the residual is $\|r\|_2 = \|c_2\|_2$.

Gram-Schmidt orthogonalization

One way to obtain the QR factorization of a matrix A is by Gram-Schmidt orthogonalization.

For each column of A, subtract out its component in the other columns.

For the simple case of 2 vectors $\{a_1, a_2\}$, first normalize a_1 and obtain

$$q_1 = \frac{a_1}{||a_1||}.$$

Now subtract from a_2 the components from q_1. This is given by

$$a_2' = a_2 - (q_1^T a_2) q_1.$$

Normalizing a_2' we have

$$q_2 = \frac{a_2'}{||a_2'||}$$

Repeating this idea for n columns gives us Gram-Schmidt orthogonalization.

Gram-Schmidt orthogonalization

Since R is upper triangular and $A = QR$ we have

$$\begin{aligned} a_1 &= q_1 r_{11} \\ a_2 &= q_1 r_{12} + q_2 r_{22} \\ \vdots &= \quad \vdots \\ a_n &= q_1 r_{1n} + q_2 r_{2n} + \ldots + q_n r_{nn} \end{aligned}$$

From this we see that $r_{ij} = q_i^T a_j, j > i$

Gram-Schmidt orthogonalization

```
function [Q,R] = gs_qr (A)

m = size(A,1);
n = size(A,2);

for i = 1:n
    R(i,i) = norm(A(:,i),2);
    Q(:,i) = A(:,i)./R(i,i);
    for j = i+1:n
        R(i,j) = Q(:,i)' * A(:,j);
        A(:,j) = A(:,j) - R(i,j)*Q(:,i);
    end
end

end
```